

Working with Large Language Models

From Tokens to Meaningful
Responses



CONTENT

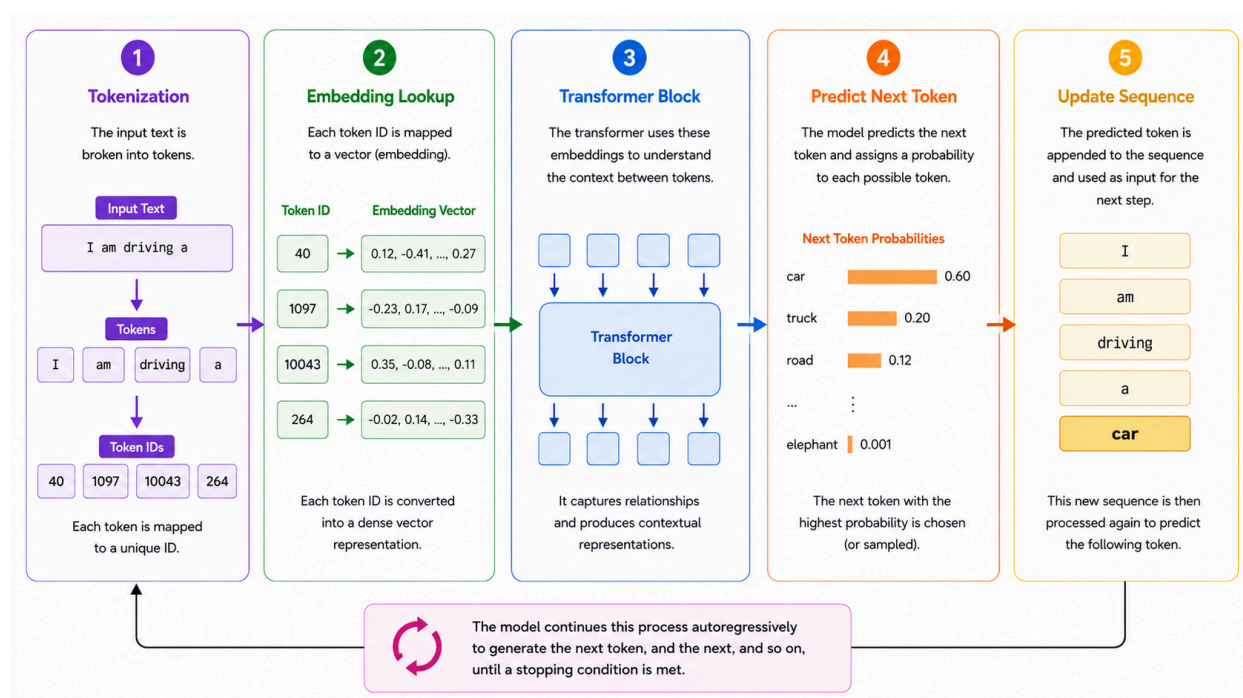
Large Language Models (LLMs)	3
1. Tokens	4
2. Embeddings	6
3. Loading the Text Generation Model	11
4. Controlling Model Output	12
4.1 Temperature	12
4.2 top_p	13

Large Language Models (LLMs)

A **Large Language Model (LLM)** is a type of deep learning model specifically designed to understand and generate human language. The name has three parts, each meaningful:

Word	What it means
Large	Billions to trillions of parameters
Language	Trained on text data - books, websites, code, papers, conversations
Model	A mathematical function that maps input to outputs

The **overall workflow of a Large Language Model (LLM)** is shown in figure below. The process begins by converting the input into tokens, which are then converted into embeddings. These embeddings are processed by the transformer block to capture the context, which enables the model to predict the next token. The predicted token is added to sequence, and the process is repeated until the desired output is generated.

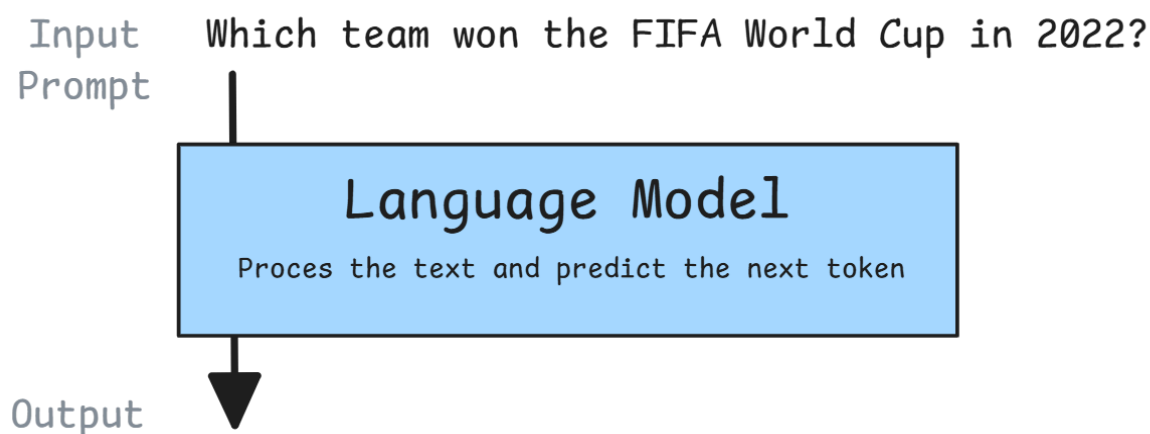


1. Tokens

A **token** is the smallest unit of text LLM can process. The token can be word, part of word (subword), punctuation, white space, emojis, etc.

The way the majority of people interact with language models is through a web playground that provides a chat interface between the user and language model. One of the good example of this is **ChatGPT**. While interacting with ChatGPT, we may notice that the model doesn't produce output response all at once, it actually generates one **token** at a time.

But tokens aren't only the output of a model, they are also the way in which the model sees an input. Viewed from outside, LLM takes input prompts and generates responses.



But before the prompt (input) is presented to the language model, it must go through the **tokenizer** that breaks it into pieces. Let's visualize it through an example of how a tokenizer breaks the input text into tokens. We will be using the tokenizer of GPT-4 on the OpenAI Platform.

GPT-5.x & O1/3
GPT-4 & GPT-3.5 (legacy)
GPT-3 (legacy)

Which team won the FIFA World Cup in 2022?

Clear
Show example

Tokens	Characters
12	42

Which team won the FIFA World Cup in 2022?

Text
Token IDs

Let's look at a code example and interact with these tokens ourselves. Here we'll be using **tiktoken** and seeing how to tokenize the input before generating text with the LLM.

```
!uv add tiktoken # need to install tiktoken
```

```
import tiktoken

enc = tiktoken.encoding_for_model("gpt-4o-mini")
token_ids = enc.encode("Which team won the FIFA World Cup in 2022?")
print(token_ids)
```

```
[34405, 3214, 3313, 290, 54571, 5922, 17257, 306, 220, 1323, 17, 30]
```

```
# These are the token ids, let's map them into words
for token in token_ids:
    print(f"{token} --> {enc.decode([token])}")
```

```
34405 --> Which
3214 --> team
3313 --> won
290 --> the
54571 --> FIFA
```



```

5922 --> World
17257 --> Cup
306 --> in
220 -->
1323 --> 202
17 --> 2
30 --> ?

```

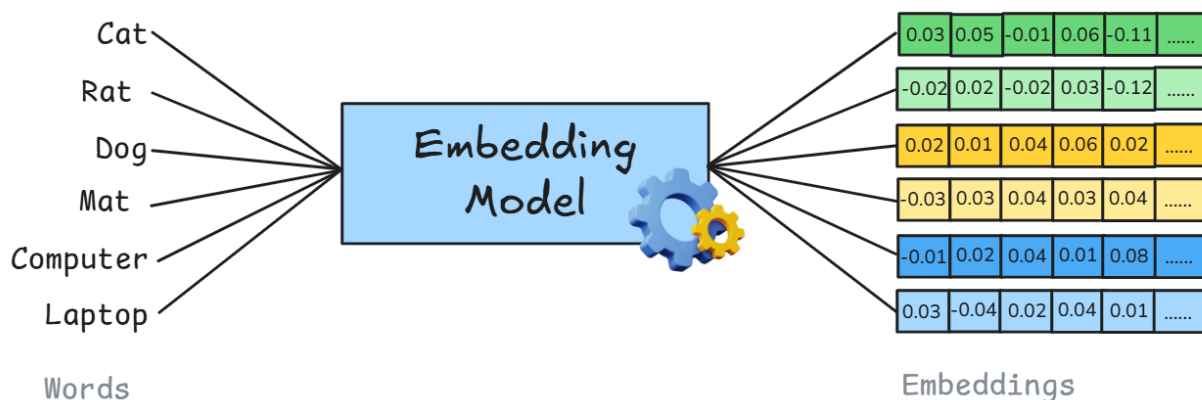
2. Embeddings

LLMs cannot directly process human-readable text. First, the text is broken into tokens using *tokenizers*. However, even the tokens are just pieces of text like “*Hel*” and “*lo*”, which the model still cannot process. Therefore, each token is converted into unique integers called *token ID*.

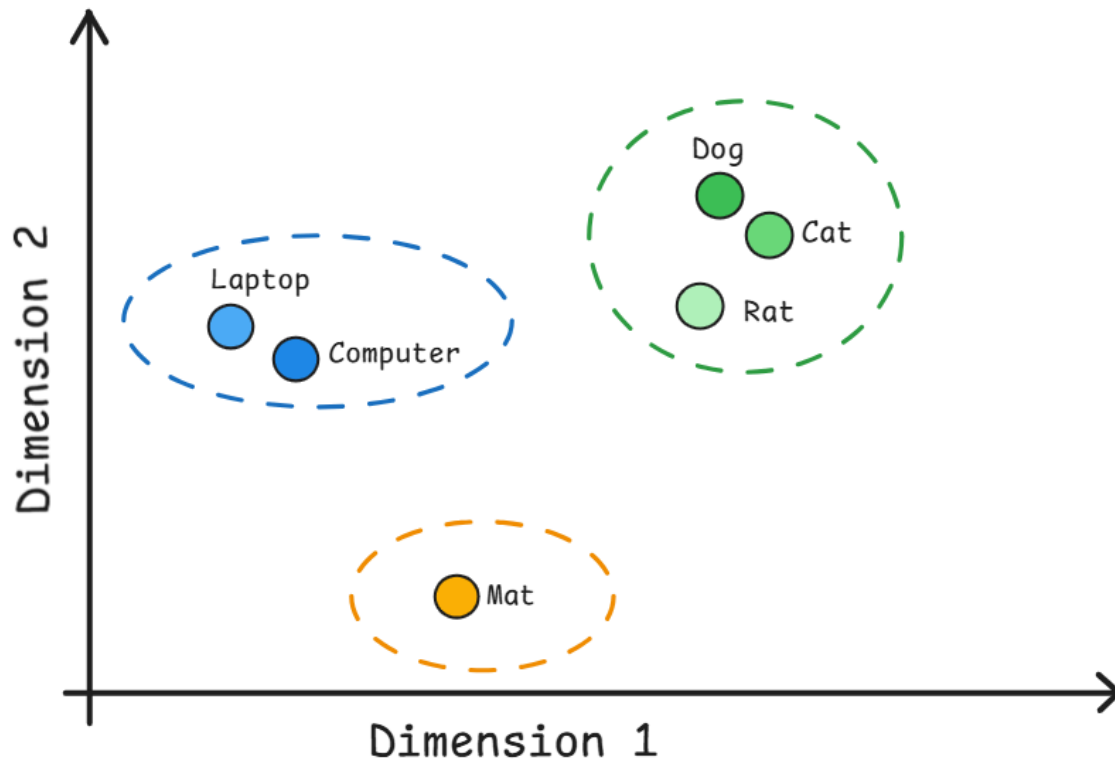
These integers themselves don't carry any meaning. So they are all passed through an *embedding layer*, which converts each token into a dense vector of numbers called an *embedding*.

Embedding is like a translator that converts the text into numerical codes. These numerical codes aren't random because similar words or items get codes that are close to each other. It's like a map where words or items with similar meaning are clustered together.

With that in mind, a more theoretical definition is that **embedding** is a dense numerical representation of data (such as text, images or audios) that are able to capture the semantic meaning. *Similar words or items have similar embeddings*.



If we project these high dimensional numerical representations into 2D space, then words that share similar meaning appear to be located near each other forming a cluster.



Let's compute the embeddings using SentenceTransformers from langchain-huggingface Python package. This tool provides a user-friendly interface, making the embedding process simple and efficient. In the code snippet below, we can see how we loaded a model from SentenceTransformers, computed the embeddings for the words, and computed the similarity between them.

```
!uv add langchain langchain-huggingface sentence-transformers numpy  
matplotlib pandas scikit-learn
```

```
from langchain_huggingface import HuggingFaceEmbeddings  
embedding_model = HuggingFaceEmbeddings(  
    model_name = "Sentence-transformers/all-MiniLM-L6-v2"  
)
```

```
# Computing the embeddings
words = ["Cat", "Rat", "Dog", "Mat", "Computer", "Laptop"]
word_embeddings = embedding_model.embed_documents(words)

import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
similarity = cosine_similarity(word_embeddings)
df = pd.DataFrame(
    similarity,
    index = words,
    columns = words
)
df
```

	Cat	Rat	Dog	Mat	Computer	Laptop
Cat	1.000000	0.498134	0.660638	0.299553	0.408935	0.289989
Rat	0.498134	1.000000	0.506694	0.301490	0.364011	0.304360
Dog	0.660638	0.506694	1.000000	0.280530	0.425002	0.347645
Mat	0.299553	0.301490	0.280530	1.000000	0.364179	0.230809
Computer	0.408935	0.364011	0.425002	0.364179	1.000000	0.714021
Laptop	0.289989	0.304360	0.347645	0.230809	0.714021	1.000000

Now, let's visualize the embeddings in 2D space. For this we will be using **PCA** so as to project the high-numerical representation into 2 dimensional vectors and then **Matplotlib**, Python's famous visualization library to plot those numbers.

```
import numpy as np
from sklearn.decomposition import PCA

word_embeddings_np = np.array(word_embeddings)
pca = PCA(n_components=3)
word_embeddings_2d = pca.fit_transform(word_embeddings_np)
word_embeddings_2d
```

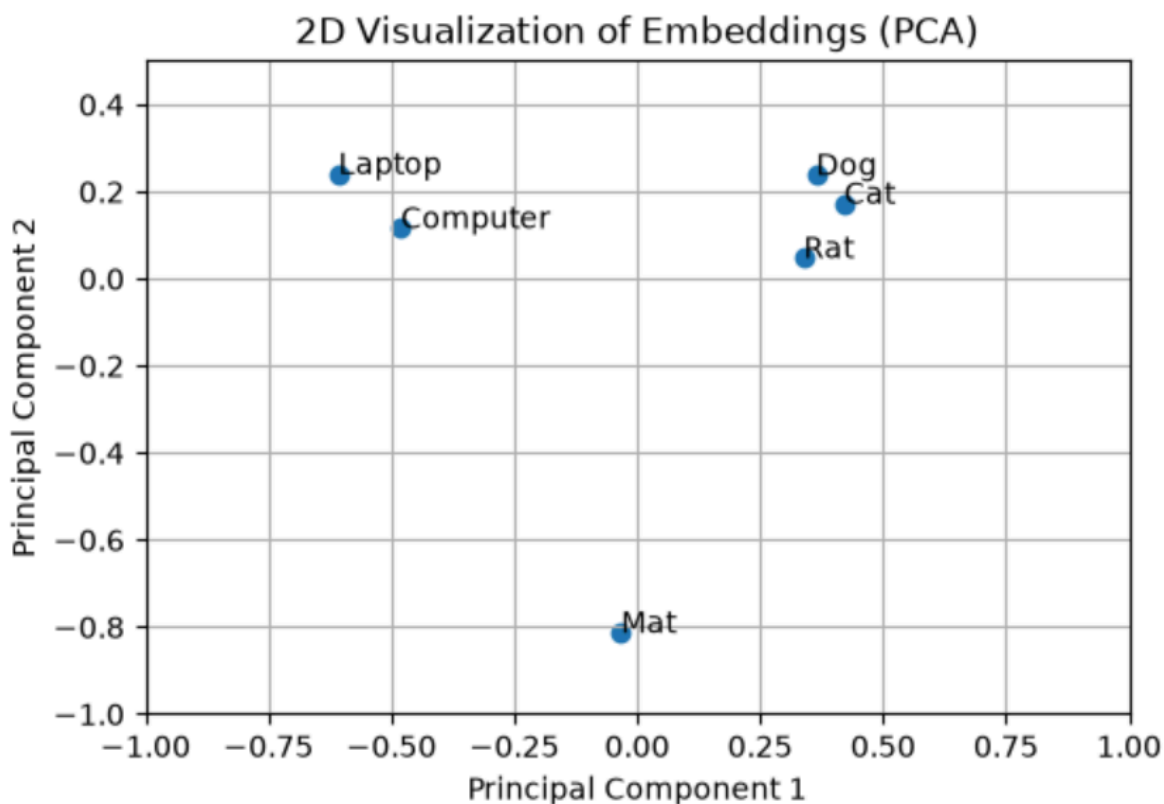


```
array([[ 0.42045823,  0.17232114],
       [ 0.33800102,  0.04729871],
       [ 0.36318468,  0.23787515],
       [-0.03298536, -0.81341884],
       [-0.48079253,  0.11559947],
       [-0.60786603,  0.24032436]])
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6,4))
plt.scatter(word_embeddings_2d[:,0], word_embeddings_2d[:,1])

# Add labels
for i, word in enumerate(words):
    plt.annotate(
        word,
        (word_embeddings_2d[i,0], word_embeddings_2d[i,1]),
    )
plt.title("2D Visualization of Embeddings (PCA)")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.xlim(-1,1)
plt.ylim(-1,0.5)
plt.show()
```



Here, we have discussed how embeddings can be computed. Due to the generative AI revolution, which uses RAG, embeddings have become extremely popular in information retrieval tasks, such as semantic search for text, code, images, and audio. But before generative AI, embeddings were already heavily used in:

- Recommender system
- Clustering & outlier detection
- Classification using embedding as features
- Zero-shot Classification by comparing the embedding of each class and picking the most similar one

3. Loading the Text Generation Model

Throughout the session, we will be using **LangChain** to interact with Large Language Models (LLMs). LangChain is a popular framework that simplifies the process of building AI-powered applications by providing an interface for loading models, creating prompts, and integrating external tools.

In this section, we will load a text generation model using LangChain:

```
!uv add langchain-openai
```

```
# Loading the environment Variables
import os
from dotenv import load_dotenv

load_dotenv(override=True)

GROQ_API_KEY = os.getenv("GROQ_API_KEY")
GROQ_BASE_URL = "https://api.groq.com/openai/v1"
```

```
# Loading the model
from langchain_openai import ChatOpenAI

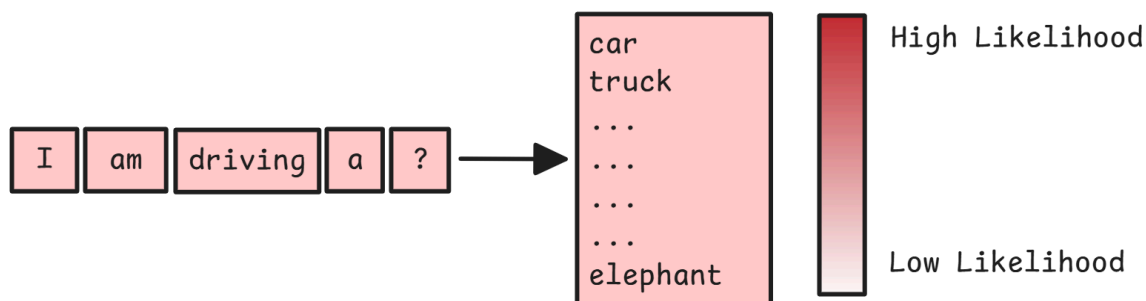
model = ChatOpenAI(
    model="llama-3.1-8b-instant", # name of the model
    api_key=GROQ_API_KEY,
    base_url=GROQ_BASE_URL
)
```

```
# generating response from model
response = model.invoke("Hello there!")
print(response.content)
```

```
Hello. How can I help you today?
```

4. Controlling Model Output

We can control the output of the model by adjusting some of the parameters. There are two parameters: **temperature** & **top_p**. These parameters control the randomness of the output.

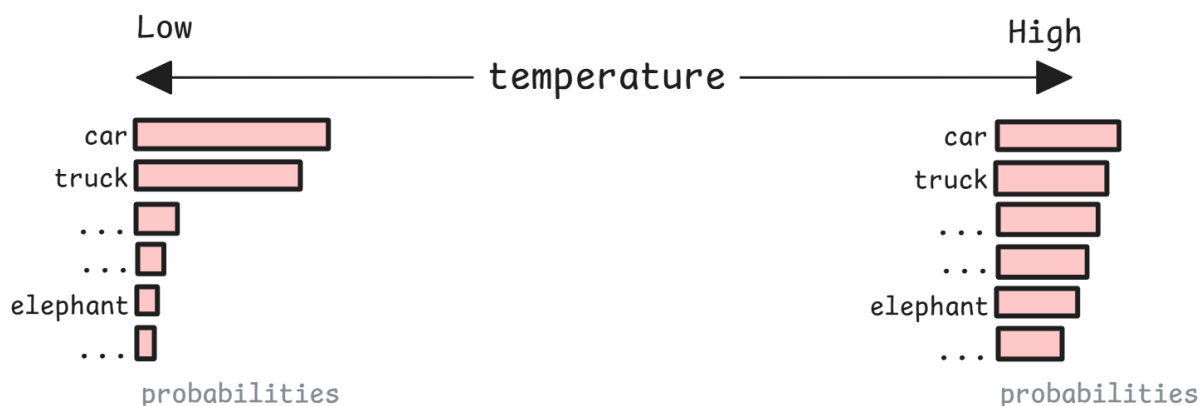


As illustrated in the figure above, in the sentence “**I am driving a...**” the likelihood of that sentence being followed by tokens like “**car**” or “**truck**” is generally higher than a token like “**elephant**”. However, there is still a possibility of “elephant” being generated but it is much lower.

4.1 Temperature

The **temperature** controls the randomness or creativity of the text generated.

- Low Temperature (e.g., 0): The model generates the same response every time because it always chooses the most likely words.
- High Temperature (e.g., 1): The model generates a more diverse response because it is more likely to choose less probable words.



We can use temperature in our above code as follows:

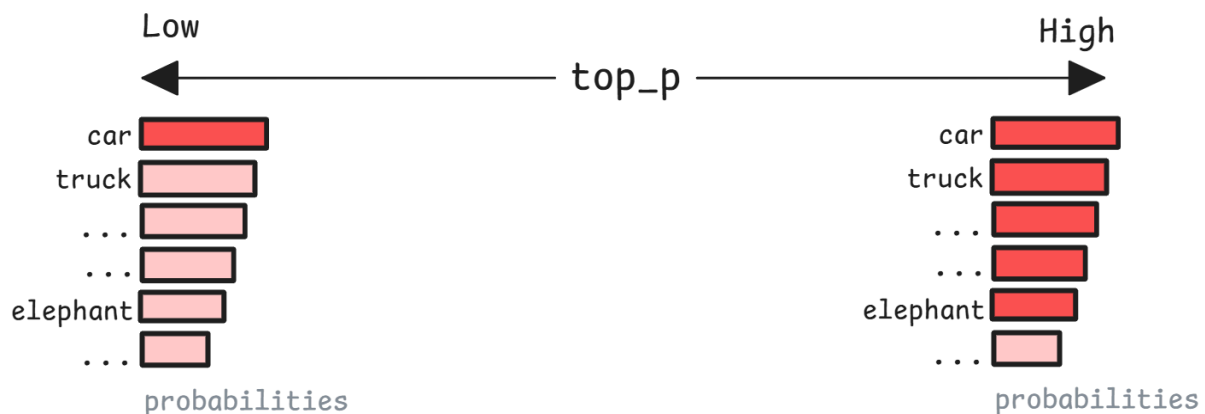
```
model = ChatOpenAI(  
    model="llama-3.1-8b-instant",  
    temperature=1.5,  
    api_key=GROQ_API_KEY,  
    base_url=GROQ_BASE_URL  
)  
  
response = model.invoke("Create a funny joke about chickens. ")  
print(response.content)
```

Why did the chicken go to therapy? Because it had a fowl mood!

Every time we rerun this piece of code, the output will change. **Temperature** introduces stochastic behavior since the model now randomly selects tokens.

4.2 top_p

The **top_p**, also known as nucleus sampling, is a sampling technique that controls which subset of a token (the nucleus) the LLM can consider. It will consider the tokens until it reaches their cumulative probability. If we set **top_p** to 0.1, it will consider tokens until it reaches that value. If we set **top_p** to 1, it will consider all tokens.



We can use **top_p** in our above code as follows:

```
model = ChatOpenAI(  
    model="llama-3.1-8b-instant",  
    temperature=2,  
    top_p = 1,  
    api_key=GROQ_API_KEY,  
    base_url=GROQ_BASE_URL  
)
```

```
response = model.invoke("Create a funny joke about chickens.")  
print(response.content)
```

Why did the chicken go to the doctor? Because it had a fowl breath!